
Sistema Automatizado de Inspeção Visual para Detecção de Defeitos em Transistores THT utilizando YOLO26

(Prof. Helton Maia, Visão Computacional 2026.1)

Laécio Miguel Albino Lopes

Julho de 2026

Abstract

Este trabalho apresenta uma Prova de Conceito (PoC) de um sistema de inspeção visual automatizada (AVI) baseado em Deep Learning para o controle de qualidade de componentes THT (Through-Hole Technology). O objetivo é desenvolver um modelo capaz de detectar e classificar o estado físico de transistores eletrônicos em imagens, identificando quebras, posicionamento incorreto e danos no encapsulamento. Realizou-se a exploração da arquitetura de rede neural convolucional YOLO26 via Transfer Learning, aplicando técnicas de Data Augmentation agressivas para mitigar a escassez de dados. Os resultados e a análise crítica da matriz de confusão demonstram o viés estatístico em classes minoritárias, evidenciando o potencial da arquitetura, bem como as limitações da generalização sob condições de desbalanceamento de dados em aplicações industriais.

1 Introdução

A automação do controle de qualidade em linhas de montagem de Placas de Circuito Impresso (PCBA) representa um pilar essencial para a manufatura eletrônica moderna. Dentro desse cenário, os componentes THT, caracterizados por terminais que atravessam mecanicamente a placa, demandam uma análise visual minuciosa quanto à sua integridade estrutural. Falhas geométricas sutis em componentes discretos, como transistores, podem comprometer a continuidade lógica e a segurança de circuitos complexos.

Técnicas tradicionais de processamento digital de imagens (PDI) mostram-se limitadas ao lidar com a complexidade e a variabilidade dos dados visuais, sendo suscetíveis a variações de iluminação e oclusões [1]. Este projeto explora o uso de Deep Learning aplicado à Visão Computacional como uma abordagem robusta, utilizando-se de redes neurais convolucionais (CNNs) para extrair características visuais relevantes e realizar a Detecção de Objetos (Object Detection) [2]. O sistema atua localizando e categorizando o estado do componente em quatro classes: *good* (íntegro), *cut-lead* (terminal cortado), *damaged-case* (encapsulamento danificado, em outras palavras, o corpo do transistor danificado) e *misplaced* (mal posicionado).

Neste trabalho, fez-se a utilização dos conceitos explorados na disciplina com o intuito de, por intermédio da Transferência de Conhecimento, treinar o modelo, focando no enfrentamento da escassez de dados através de técnicas modernas de regularização e aumento de dados, culminando em uma análise rigorosa do comportamento do modelo perante dados desbalanceados.

2 Desenvolvimento

2.1 Conjunto de Dados

Os dados brutos utilizados pertencem ao *Transistor Dataset* (versão 2), disponibilizado publicamente via Roboflow Universe. O volume original compreende 166 imagens pré-anotadas no formato YOLO. As classes presentes são: `good`, `cut-lead`, `damaged-case` e `misplaced`.

Na origem, o dataset foi submetido a um pré-processamento estático que incluiu auto-orientação, redimensionamento (*Stretch*) para 416×416 pixels e conversão monocromática (*Grayscale - CRT phosphor*) para simplificação dimensional do tensor de entrada. Adicionalmente, técnicas de aumento prévia aplicaram recortes aleatórios (0-23%), ajuste de brilho ($\pm 35\%$) e suavização gaussiana. O conjunto foi particionado estruturalmente em diretórios de treinamento (*train*), validação (*val*) e teste (*test*).

2.2 Arquitetura do Modelo

Explorou-se arquiteturas de Deep Learning voltadas para detecção de objetos de alta performance:

1. **Rede Neural Convolucional (CNN):** As camadas convolucionais extraem características visuais hierárquicas, mapeando primitivas de baixo nível (bordas e gradientes) em representações abstratas complexas.
2. **Transfer Learning e YOLO26:** Utilizou-se a arquitetura YOLO26 (versão *Nano - yolo26n.pt*). Esta arquitetura opera sob a filosofia *edge-first*, introduzindo uma cabeça de detecção *One-to-One* (NMS-free), eliminando o pós-processamento de Supressão de Não-Máximos, o que otimiza a latência e a precisão em alvos de pequenas dimensões.

No aprendizado supervisionado de redes convolucionais, a otimização dos parâmetros da rede minimiza o erro escalar através do algoritmo de retropropagação. A formulação matemática clássica para a função de perda (*Cross-Entropy*) na etapa de classificação da rede é representada por:

$$L = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{i,c} \log(\hat{y}_{i,c}) \quad (1)$$

onde:

- L é a perda média sobre todas as amostras
- N é o número de amostras
- C é o número de classes (no caso, $C = 4$)
- $y_{i,c}$ é o rótulo verdadeiro (0 ou 1) da i -ésima amostra para a classe c
- $\hat{y}_{i,c}$ é a probabilidade predita pelo modelo

2.3 Implementação

A implementação e o treinamento foram realizados na nuvem (Google Colab) com aceleração de hardware (GPU Nvidia Tesla T4), utilizando a biblioteca `ultralytics` em Python.

Diante do tamanho diminuto do dataset, o modelo apresentava risco de sofrer *overfitting* memorizando ruídos em vez de generalizar padrões. Para contornar esse desafio empírico, injetou-se hiperparâmetros agressivos de **Data Augmentation dinâmico** durante as épocas de treino:

- **Mosaico e MixUp:** Combinação de 4 imagens em uma única matriz (`mosaic=1.0`) e mesclagem de rótulos (`mixup=0.1`).
- **Perturbações Espaciais e Cromáticas:** Rotação (`degrees=10`), escala (`scale=0.3`), distorção de cisalhamento (`shear=5`), e variações no espaço de cor HSV.

- **Otimização:** Utilizou-se lotes reduzidos (`batch=8`), limite de 100 épocas e *Early Stopping* (`patience=20`) para prevenir a degradação da validação. O modelo final foi exportado e embarcado em uma aplicação *web* utilizando a biblioteca **Gradio** para permitir inferência em tempo real.

3 Resultados

3.1 Métricas de Avaliação e Desempenho

Utilizou-se métricas de destaque no âmbito de Detecção de Objetos (Precision, Recall e Mean Average Precision - mAP). A Tabela 1 consolida o desempenho obtido pelo modelo YOLO26n no subconjunto de validação formal (13 instâncias):

Table 1: Resultados de Desempenho por Classe no Conjunto de Validação

Classe	Amostras (Instances)	Precisão (P)	Revocação (R)	mAP@50
<i>Geral (Todas)</i>	13	0.627	0.885	0.868
cut-lead	1	0.514	1.000	0.995
damaged-case	3	0.413	0.667	0.617
good	8	0.722	0.875	0.867
misplaced	1	0.860	1.000	0.995

A alta Revocação (88.5%) indica a eficiência da rede em não omitir os defeitos estruturais (baixos índices de falsos negativos). A convergência da rede também pode ser confirmada observando o progresso das perdas e inferências visuais geradas:

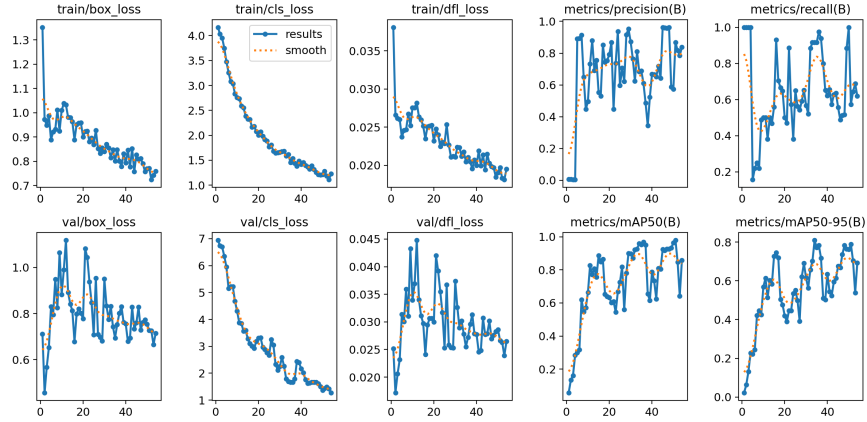


Figure 1: Curvas de convergência de Loss e evolução de mAP ao longo do treinamento.

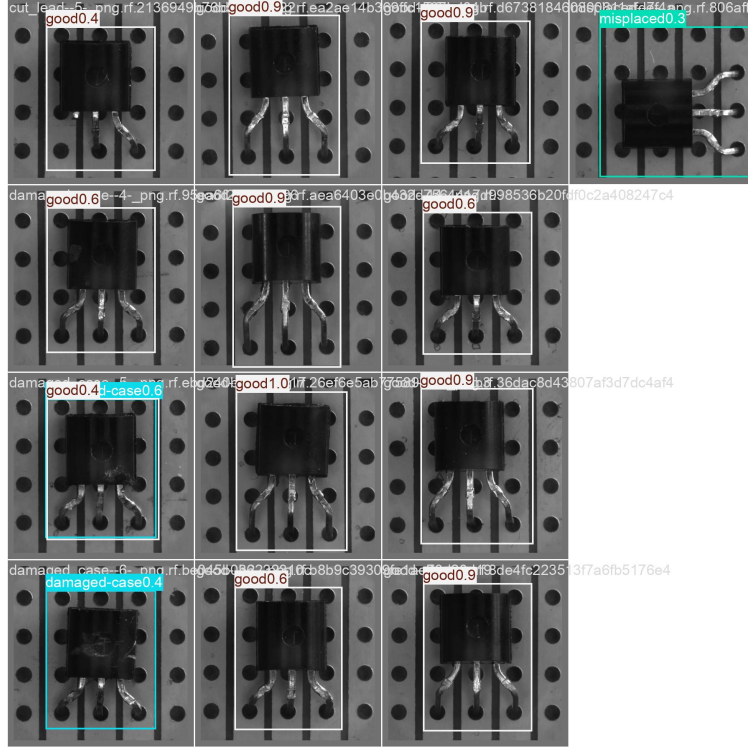


Figure 2: Inferência visual do modelo sobre o lote de controle de validação.

3.2 Discussão e Análise Crítica

Apesar dos excelentes resultados matemáticos globais na validação, testes práticos com imagens capturadas fora do domínio do dataset revelaram o fenômeno do viés estatístico. Analisando a matriz de confusão (Figura 3), nota-se um profundo desbalanceamento: a classe nominal **good** detém 8 amostras de controle, enquanto classes fundamentais de defeito como **cut-lead** possuem apenas 1.

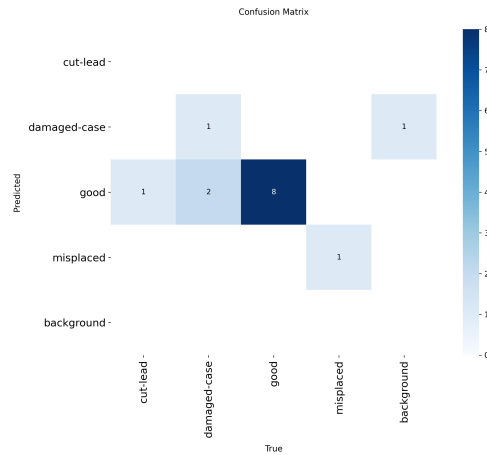


Figure 3: Matriz de Confusão demonstrando o severo desbalanceamento amostral.

Durante testes de inferência externa no simulador desenvolvido via Gradio, notou-se a dificuldade do modelo em generalizar perante novas condições de iluminação. Ao reduzir o limiar de confiança (*confidence threshold* para 15%), a rede manteve precisão formidável na *localização convolucional* (desenhando a *bounding

box* corretamente), porém manifestou confusão na etapa de *classificação*, tendendo a classificar componentes com terminais visivelmente cortados como *good*.

Tal comportamento corrobora empiricamente a teoria de *Overfitting* a classes majoritárias: diante da incerteza visual, o modelo favorece a classe estatisticamente dominante em seu treinamento, expondo que o *Data Augmentation* computacional mitiga, mas não substitui, a necessidade fundamental de um volume de dados nativos robusto e balanceado [1].

4 Conclusão

Por conseguinte, neste trabalho, atingiu-se o objetivo de expor o uso de Visão Computacional para a inspeção de componentes THT. Treinou-se e avaliou-se um modelo baseado na arquitetura YOLO26, implementando estratégias rigorosas para contornar a escassez de dados. A aplicação interativa provou o conceito e a viabilidade da técnica para ambientes industriais. Contudo, a análise crítica evidenciou que a maturação deste sistema demanda futuras investigações centradas na coleta de datasets proprietários balanceados ou na adoção de funções de custo penalizadoras (como *Focal Loss*), evidenciando que a qualidade do dado e, também, a amostragem são aspectos tão cruciais quanto a modernidade da arquitetura de detecção.

5 Repositório de Código

O código deste projeto (treinamento e interface web Gradio) está disponível em:

- GitHub: <https://github.com/laecyomiguel/projeto-final-vc>
- Google Colab: <https://colab.research.google.com/drive/1QqSnF3sA9wjqbIUxc1x1rawh3mqNQhGx?usp=sharing>

References

- [1] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep learning*. MIT press, 2016.
- [2] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521(7553):436–444, 2015.